

complexity.

Common Mapping Schema

Mapping/look-up (map) tables to support dimension tables should be created in the commonmapping schema.

Common Schema

The Common schema is where all of the facts and dimensions that compose the Enterprise Dimensional Model are stored. The Common schema contains a variety of different types of dimensions and facts that create multiple star schemas. The models in this schema are robust and provide the basis for analyzing GitLab's businesses processes.

Our dimensional model revolves around two core components: dimensions (providing context) and facts (measuring events). Understanding each component's role and characteristics is crucial for effective data modeling.

Dimension Tables

Dimension tables supply the descriptive attributes that give context to our business events. These are the "who, what, where, when, why, and how" of our data. A dimension table typically represents a business entity like a customer, product, or location.

Characteristics of Dimensions

- Contain descriptive attributes (text-based or categorical)
- Usually have fewer rows than fact tables but more columns
- Change relatively slowly over time
- Provide the entry points for querying and filtering
- Include hierarchical relationships

Common Types of Dimensions

Conformed Dimensions Conformed dimensions enable facts and measures to be consistently categorized and described across multiple fact tables and data marts. These dimensions maintain consistent meaning across multiple fact tables. This standardization ensures reliable analytical reporting and promotes reusability throughout the data warehouse. When properly implemented, each subject area can be analyzed independently while maintaining the ability to combine insights with related areas. However, this cross-area analysis capability breaks down if dimensions have even slight variations between subject areas. Kimball refers to this standardized set of shared dimensions as the "conformance bus", which facilitates seamless integration of common dimensions and enables comprehensive reporting across multiple subject areas. Examples include:

- Date dimensions used across various business processes
- Customer dimensions used in sales and marketing

Local Dimensions Specific to a single business process or fact table, these dimensions provide context for particular events or metrics. Examples include:

- Support ticket status
- Order types
- Campaign attributes

Slowly Changing Dimensions & Snapshots

Understanding Time Perspectives

Data analysis typically requires two viewpoints: current and historical. The current view uses up-to-date dimension values, while historical analysis needs to understand how things looked at specific points in time. For example:

- Analyzing sales with a previous product catalog
- Tracking customer location changes over time
- Understanding organizational structure changes

Types of Dimensions

We implement three approaches to handle time-based changes:

Type 1 Dimensions

- Overwrite values when they change
- Maintain only current state
- Provide simplest implementation
- Lose historical context

Type 2 Dimensions (SCD)

- Add new records for changes
- Track validity periods with validfrom and validto dates
- Enable historical analysis
- Maintain complete change history

Type 3 Dimensions

- Maintain current and alternate values
- Enable multiple analytical perspectives
- Support dual categorization needs
- Not currently implemented in our EDM

SCD in Practice

Snapshot tables form the backbone of our historical tracking system, capturing the complete lifecycle of business objects from creation through every modification to the present state. These tables maintain a detailed audit trail of changes while enabling efficient historical analysis.

In our implementation, Slowly Changing Dimensions are created using dbt's snapshot functionality. dbt snapshots provide a simple yet powerful way to track historical changes in

our data. When a snapshot is run, dbt compares the current state of the data with the previous snapshot and automatically tracks any changes through validfrom and validto dates.

A snapshot tracks changes through validity periods, marking each state with these timestamps. For example, a simple state change might look like:

id	attribute	validfromdate	validtodate
----	-----	-----	-----
1	'open'	2022-01-01	2021-01-02
1	'closed'	2022-01-02	NULL

While this format efficiently stores historical data, it can be challenging for business users to analyze. To improve usability, we transform these snapshots into daily grain records in the COMMON schema. These dailysnapshot models expand the validity periods into individual day records, making time-based analysis more intuitive while maintaining consistency with our dimensional model.

During model development, we start with staging models in COMMONPREP to implement business logic, then use dbt's snapshot functionality to track changes. This foundation allows us to create daily snapshots when needed while managing performance impacts.

Best practices for snapshot implementation include adding clear current record indicators, maintaining consistent grain across related models, and thoroughly documenting validity periods. These practices ensure our historical tracking remains accurate and performant while serving diverse analytical needs.

Fact Tables

Fact tables record the business events we want to analyze. They contain the quantitative metrics (measures) of our business processes, along with references to related dimensions.

Key Characteristics

Fact tables capture business events with numeric measures and typically grow continuously as new events occur. They're characterized by many rows (each representing a single event) but relatively few columns.

While most measures in these tables can be aggregated, some can only be partially aggregated (like averages), and others shouldn't be aggregated at all (like percentages). Each fact table represents a specific business process and includes foreign keys that link to dimension tables, providing context to the measures.

Types of Facts

Atomic Facts Fact tables form the foundation of fact-based analysis by capturing business events at their most granular level. Each row represents an individual business event with complete, unfiltered data, preserving the maximum level of detail. By maintaining this granularity, atomic facts enable flexible aggregation options for various analytical needs.

Derived Facts Derived facts are specialized views built on top of atomic facts to serve

specific analytical needs while maintaining clear data lineage. They improve performance by creating focused subsets of large atomic fact tables. For instance, when analysts typically work with just 10% of a large event table, a derived fact can extract just that portion, improving query speed. These tables also standardize metrics by precomputing commonly used aggregations.

Additionally, through "drill across facts", we can combine multiple fact tables through their shared conformed dimensions to create unified analytical views. By using full outer joins on common dimensions, this process creates derived fact tables that link related business processes while maintaining dimensional consistency, enabling analysis across multiple business areas.

Special Purpose Tables

Bridge Tables

Bridge (bdg) tables reside in the common schema and serve a crucial role in our dimensional model. These intermediate tables resolve many-to-many relationships between tables, maintaining data model flexibility while ensuring proper relationship management.

Common Mart Schema

The Common Mart schema combines dimensions and facts into business-ready analytics models, serving as the primary access point for business users and analytics tools.

Purpose and Structure

The mart layer transforms our dimensional model into business-specific datasets that combine relevant facts and dimensions. By pre-joining common attributes and applying standard business rules, marts optimize data for specific analytical needs within each business domain.

Organization By Business Domain

Mart models are organized by business function, separating data into focused areas like Finance, Marketing, People, Product, and Sales. This functional organization allows each department to work with data that's specifically tailored to their analytical needs while still maintaining connections to the broader data warehouse through conformed dimensions.

Key Characteristics

Mart models are built directly on fact and dimension tables from the Enterprise Data Model, ensuring consistent business definitions and standardized relationships. To maintain data lineage and prevent complexity, they never reference other mart models as sources.

Designed with analytics in mind, these models feature pre-joined tables for common query patterns and include frequently used calculations. The data grain is carefully considered to balance user needs with performance requirements.

The design puts business users first by using familiar business terms for tables and fields, rather than technical language. This business-oriented structure and documentation supports

self-service analytics by aligning with common analysis patterns, making the data more accessible and intuitive.

Best Practices

- Keep models focused on specific business domains
- Document assumptions and limitations
- Maintain consistent naming across related marts
- Regular testing of business logic
- Monitor usage patterns for optimization
- Maintain clear lineage to source models

Special Purpose Tables

Scaffold Tables

Scaffold tables serve as a foundational structure between fact tables, ensuring comprehensive coverage of all possible dimensional combinations in visualizations and analyses. When working with visualization tools like Tableau, these tables become particularly valuable by filling in gaps and enabling consistent comparisons between actuals and targets. They're especially useful for time-based analysis, where you need to maintain a continuous view even when data points are missing. By providing this complete dimensional framework, scaffold tables help prevent misleading gaps in reports and dashboards, ensuring that analysts can see the full picture, including periods or combinations where no data exists. These scaffold tables reside in the commonmart schema with the rptscfild prefix, building on top of fact tables while maintaining complete dimensional combinations.

In our targets vs actuals reporting, we use a scaffold table to ensure data completeness and consistency. This table maintains a comprehensive structure that covers all time periods and attribute combinations, whether or not sales activity occurred.

Specific Schema

The SPECIFIC schema is to be used for tables that perform a reporting function and act as a source of truth but do not conform to the dimensional modeling structure of the Enterprise Dimensional Model.

No Transformaion Views

A No Transformation View should be direct views of raw source data that are needed for reporting without further transformation. They should not be used to build additional tables since there will be a table upstream in the RAW or PREP database that will provide better lineage documentation for further transformations. They should always be created as a view with no additional transformation or filtering and should be prefixed with ntv.

Entitlement

To facilitate the use of row level security in both Snowflake and Tableau a schema dedicated for entitlement tables, a mapping between the user or role and the records they are allowed to see, is used. The tables in this schema follow a standard form but are not limited to an exact

structure. The purpose of these tables is to be joined to other tables in such a way that at query time the second table will be limited to the appropriate records for the runner of the query.

Naming

The name of the entitlement table should direct users to the other table or tables that it should be used in combination with as well as the application it should be used with. Documentation for exactly what tables the entitlement table should be used for can be found in the data warehouse model documentation. For example an entitlement table that would be used with the martteammemberdirectory table in Tableau would be named entteammemberdirectorytableau.

Form

Each entitlement table must have at least two columns: a join key that will connect to an other table and represents a subset of records and a column representing a Tableau user or Snowflake role. The column that is used to join to an other table should be named the same as it is in that table to make it easier to user the correct table. The values in the column that is used as a join key should represent the values of that column in the corresponding table. The column or columns that represent the Tableau users or Snowflake roles should be named to match.

Every combination of user and join key must be explicitly included in as row in the table.

Example

If row level security is to be implented on the martteammemberdirectory table diretly in Snowflake then the following table would be created:

entteammemberdirectorysnowflake

costcenter	snowflakerole
Cost of Sales	TMEMBER1
G&A	TMEMBER1
R&D	ANALYSTGROUP
Marketing	ANALYSTGROUP

Then a row access policy would be applied to the martteammemberdirectory table.

If row level security is to be implented on the martteammemberdirectory table in a Data Source in Tableau then the following table would be created:

entteammemberdirectorytableau

costcenter	tableauuser
Cost of Sales	teammember1@gitlab.com
Cost of Sales	teammember2@gitlab.com
Cost of Sales	teammember3@gitlab.com
G&A	teammember1@gitlab.com

G&A	teammember2@gitlab.com
Sales	teammember1@gitlab.com
R&D	teammember3@gitlab.com
Marketing	teammember1@gitlab.com

Then, using the guidelines outlined in Tableau Developers Guide, a Data Source and filters would be created.

Technical Implementation Details

Naming Standards

It is critical to be intentional when organizing a self-service data environment, starting with naming conventions. The goal is to make navigating the data warehouse easy for beginner, intermediate, and advanced users. We make this possible by following these best practices:

1. PREP TABLES: prep<subject> = Used to clean raw data and prepare it for dimensional analysis.
1. FACT TABLES: fct<verb> Facts represent events or real-world processes that occur. Facts can often be identified because they represent the action or 'verb'. (e.g. session, transaction)
1. DIMENSION TABLES: dim<noun> = dimension table. Dimensions provide descriptive context to the fact records. Dimensions can often be identified because they are 'nouns' such as a person, place, or thing (e.g. customer, employee) The dimension attributes act as 'adjectives'. (e.g. customer type, employee division)
1. MART TABLES: mart<subject> = Join dimension and fact tables together with minimal filters and aggregations. Because they have minimal filters and aggregations, mart tables can be used for a wide variety of reporting purposes.
1. REPORT TABLES: rpt<subject> = Can be built on top of dim, fact, and mart tables. Very specific filters are applied that make report tables applicable to a narrow subset of reporting purposes.
1. PUMP TABLES: pump<subject> = Can be built on top of dim, fact, mart, and report tables. Used for models that will be piped into a third party tool.
1. MAP TABLES: map<subjects> = Used to maintain one-to-one relationships between data that come from different sources.
1. BRIDGE TABLES: bdg<subjects> = Used to maintain many-to-many relationships between data that come from different sources. See the Kimball Group's documentation for tips on how to build bridge tables.
1. SCAFFOLD TABLES: rptscfild<subject> = Used to support the visualization layer by creating a template / blueprint with all the combinations of common dimensions between the desired fact tables.
1. Singular naming should be used, e.g. dimcustomer, not dimcustomers.
1. Use prefixes in table and column names to group like data. Data will remain logically grouped when sorted alphabetically, e.g. dimgeolocation, dimgeoregion, dimgeosubregion.
1. Use dimension table names in primary and foreign key naming. This makes it clear to the user what table will need to be joined to pull in additional attributes. For example, the primary key for dimcrmaccount is dimcrmaccountid. If this field appears in fctsubscription, it will be named dimcrmaccountid to make it clear the user will need to join to dimcrmaccount to get additional account details.
1. Dimension, fact, and mart tables are not to contain references to operational systems. We abstract the name away from the source system the data is produced into a name that describes the business entity or semantic significance of the data. For example, data from Salesforce is

described as crm in the dimensional model and not sfdc or salesforce.

File-Based Data Sources

When handling non-CSV data sources, we prefer direct extraction from source systems. However, temporary solutions using seed files, Sheetload, or Driveload may be acceptable with a clear deprecation plan. For CSV-type source data, we have three options:

1. dbt seed

- Version controlled via GitLab
- Easy updates via MR
- Best for < 1,000 rows
- Preferred for small datasets

1. GCP Driveload

- Stable and predictable
- No unexpected changes
- Requires manual file updates
- Preferred for larger datasets

1. Sheetload

- Enables team data entry
- Less stable implementation
- Difficult SOX compliance
- Last resort for Tier 1 assets

Testing Framework

Models require testing and documentation via schema.yml files, following the Trusted Data Framework (TDF).

Time Standards

We standardize on Monday as the first day of week across all systems:

```
sql
CASE WHEN dayname = 'Mon' THEN dateday
      ELSE DATETRUNC('week', dateday)
END AS firstdayofweek
```

This ensures:

- Consistent weekly reporting
- Alignment with ISO 8601
- Accurate metric calculations
- Standard implementation across models

Entity Relationship Diagram (ERD) Library

These diagrams provide the relationships between data objects in the Enterprise Dimensional Model across the major business process fly wheels.

Lead to Cash ERDs

<details markdown=1>

<summary>ERD Library</summary>

- Sales Funnel ERD
- Annual Recurring Revenue (ARR) ERD
- Common Subscription Model ERD
- Common Behavior ERD
- Delta ARR: Subscription Lineage Monthly ERD
- DRAFT: Quota ERD
- DRAFT: Orders ERD

</details>

Product Release to Adoption ERDs

<details markdown=1>

<summary>ERD Library</summary>

- Common Product Usage Data Model ERD
- Common Behavior ERD

</details>

Team Member ERDs

<details markdown=1>

<summary>ERD Library</summary>

- Common Team Member Data Model ERD
- DRAFT: Recruiting ERD

</details>

Create Entity Relationship (ER) Diagrams using Lucidchart

Lucidchart is a web-based diagramming application that allows users to visually collaborate on drawing, revising and sharing charts and diagrams, and improve processes, systems, and organizational structures.

An Entity Relationship (ER) Diagram is a type of flowchart that illustrates how entities or

objects relate to each other within a system. ER diagrams are used to model and design relational databases, in terms of logic and business rules (in a logical data model) and in terms of the specific technology to be implemented (in a physical data model.)

The below steps illustrate how to create an ER Diagram (Logical & Physical Data Model) using Lucidchart's ERD import functionality. This process of creating a data model from a database or a script is also known as Reverse engineering.

<details markdown=1>

<summary>Step 1: Create a blank lucid document from 'Lucidchart app' (should be available via Okta dashboard in case a user is assigned to it).</summary>

</details>

<details markdown=1>

<summary>Step 2: Click 'Import Data' that appears below the 'Shape Library' located at the bottom left hand side of the page. </summary>

</details>

<details markdown=1>

<summary>Step 3: Select 'Entity Relationship (ERD)' from 'All Data Sources'. And choose 'Import from SQL Database' option from the dropdown list menu of 'Import your Data'.</summary>

</details>

<details markdown=1>

<summary>Step 4: Select 'MySQL' as the DBMS source to import the data from, to create the ERD.</summary>

</details>

<details markdown=1>

<summary>Step 5: Run the below script in Snowflake after updating/modifying the filter criteria for selecting the 'tableschema' for selecting the tables for which the ERD needs to be created for.</summary>

sql

The below query can be run in Prod database in Snowflake to get all the Models/tables from COMMON and COMMONPREP Schemas:

```
SELECT 'mysql' dbms,  
       t.TABLESCHEMA,  
       t.TABLENAME,  
       c.COLUMNNAME,  
       c.ORDINALPOSITION,
```

```
c.DATATYPE,
c.CHARACTERMAXIMUMLength,
" CONSTRAINTTYPE,
" REFERENCEDTABLESCHEMA,
" REFERENCEDTABLENAME,
" REFERENCEDCOLUMNNAME
FROM INFORMATIONSCHEMA.TABLES t
LEFT JOIN INFORMATIONSCHEMA.COLUMNS c
ON t.TABLESCHEMA=c.TABLESCHEMA
AND t.TABLENAME=c.TABLENAME
WHERE t.TABLESCHEMA NOT IN('INFORMATIONSCHEMA')
AND t.TABLESCHEMA IN ('COMMON', 'COMMONPREP')
```

</details>

<details markdown=1>

<summary>Step 6: Download and export the results/output of the above query from Snowflake in to a csv file.</summary>

</details>

<details markdown=1>

<summary>Step 7: Navigate back to Lucidchart app and select the result.csv file to be uploaded and click 'Import'.</summary>

</details>

<details markdown=1>

<summary>Step 8: All the tables/models from the selected 'tableschema' list will now appear under 'ERD Import'. The schemas can be expanded and scrolled through to view all the tables.</summary>

</details>

<details markdown=1>

<summary>Step 9: The required tables/entities of interest can be dragged on to the canvas and relationships between the entities can be defined from the ribbon to create the ER Diagram.

Note: The number of fields to be shown for each of the entity can easily be modified from the 'Advanced Options' section that appears towards the right hand side of the page.</summary>

</details>

Big Data

Big Data is a concept that we use to understand the limits of data offerings. Generically, Big Data is anything that exceeds or strains our current technical capacity for processing and delivery. Dealing with Big Data may be less efficient and more costly as new or creative solutions need to be developed and deployed to expand the capabilities of the data offerings.

Big Data and the Enterprise Data Warehouse

Big Data in the Enterprise Data Warehouse is categorized by three general topics:

- 1. Volume - how much data there is for a relevant source, concept, or model
- 1. Velocity - how quickly the data changes, is ingested, or consumed
- 1. Variety - how the structure and format of a data source compares to other sources

The Enterprise Data Warehouse is limited in each of these conceptual areas and any data that approaches these limits would be considered Big Data.

Volume

While there is not a real limit to the amount of data that can be stored in the Enterprise Data Warehouse there are limits to how much data can be transformed in a performant and cost effective way. The current limits are being able to create a single table within a 3 hour timeframe using an XL snowflake warehouse. As an example all of the collected snowplow data, over 6TB and 30 billion records, can not all be processed at once and would be considered Big Data.

Velocity

As the Enterprise Data Warehouse is designed to process and transform the data to present a curated set of tables, there are limits to how quickly those transformations can be processed in a cost effective way. The Enterprise Data Warehouse is designed to process the data every 24 hours and data that needs to be processed more quickly than that to accommodate business needs would be considered Big Data. As an example evaluating website behavior in near real time would be considered Big Data.

Variety

The current design of the Enterprise Data Warehouse is build on the Snowflake cloud database. This limits the formats and structures of data that can be processed to those that fit into structured tables. While some processing of semi-structured data, such as JSON, is possible directly in the Enterprise Data Warehouse this is limited and must be first ingested as a column in a table decreasing efficiency. Generally, any data input or output from the Enterprise Data Warehouse that is not in a structured table would be considered Big Data. As an example the service ping payload from the Version database is JSON that requires extensive manipulation before in can be analyzed and would be considered Big Data.

Analytics Performance Policy Framework

Problem Statement

Due to increasing data volumes and business logic complexity in the Enterprise Data Warehouse, the data model transformations built in the EDW have become increasingly non-performant overtime with the daily dbt model production run taking over 12 hours to complete. Query runtimes on some of our largest Snowplow, Service Ping, and GitLab.com data sets can take longer than several minutes to complete on L and XL size warehouses. Our daily Snowplow event data volumes are expected to increase by 2.5x within the next 12+ months. Therefore, we need an Analytics Performance Policy that can provide guidelines on how to architect performant data models in the transformation layer of the EDW that balance technical considerations with business needs and requirements.

We think about dbt model runs along 3 major dimensions: performance, efficiency, and cost.

1. Performance relates to how long it takes a model to build
1. Efficiency relates to how well a model uses local storage, remote storage, and partition pruning.
1. Cost relates to how many Snowflake credits are required to run a model and is impacted by both the performance and efficiency of the model.

The scope of this Analytics Performance Policy at this time is specifically focused on the performance of models. In the future, we will consider adding a separate efficiency and cost policy that would roll-up to an overall Analytics Scalability Policy.

The Analytics Performance Policy is only considering the data transformations and does not consider retention of data that is extracted and loaded towards the RAW database of the EDW. For the time being, the policy assumes we will keep all data in the RAW database and we will not delete data. After a data retention policy is implemented in the future, we would reevaluate and iterate on this Analytics Performance Policy that focuses on the Transformation layer of the EDW. The alignment that is reached with the Functional Teams in this Analytics Performance Policy will be used to influence a data retention policy on the RAW database in Snowflake.

Performance Targets

These initial performance targets were created to allow the daily dbt model production run to finish within an 8 hour working day and provide for the run to be triaged within a working day. The Snowflake query time targets were created to make incremental improvements from several minutes query times to 1 minute to provide for a more productive and delightful querying experience in Snowflake. These targets are subject to change in the future as we continue to improve performance and receive new business requirements.

1. Maintain production dbt DAG run time of less than 8 hours. Assumes we do not scale up warehouse size for existing models and keep using a XL warehouse as a maximum. Assumes we can scale out with using more concurrent threads running at the same time.
1. An individual dbt model's run time is consistently less than 1 hour, and its design accounts for forecasted data volume increases.
1. A simple query of the Snowplow, Service Ping, and GitLab.com big data sets in Snowflake finishes in under 1 minute on a L or XL warehouse.

Architectural Approaches to Improve Performance

1. Transform and Surface Smaller Amounts of Data in the Data Models.

1. Limit the amount of rows and/or columns surfaced in the atomic fact tables (lowest grain that captures ALL transactions for a business process), based on business needs weighed against technical and performance constraints. Ex. Limit the data in a model to the last 13 months of data that is both performant and required for business analysis.

1. Shard monolithic atomic fact tables (lowest grain that captures ALL transactions for a business process) that model the higher levels of abstraction of a business process into smaller data models that model sub-business processes. Ex. Model the SDLC (Software Development Lifecycle) into its component parts like SCM, CI, CD, Security etc. versus having the end to end SDLC modeled in one, monolithic, super large data model.

1. Create aggregated data models at a level of detail purpose built for Business Analytics use cases. Ex. Consider aggregating product usage data to the user, namespace, installation, metric grains by week, month, quarter, and year timeframes as required by the business use case.

1. Consider adding a clustering key to the data model that is aligned to commonly queried use cases in the model.

1. Evaluate the use of the simplecte macro in the data model and insure only the required columns that are needed for the model are selected.

1. Consider setting the data model to be incremental.

Historical Archiving Process

1. For non-idempotent data, which is data that cannot be recreated or otherwise surfaced in the data model due to a performance policy consideration, leverage a data platform archiving methodology to create an historical archive of the data.

For example, with only exposing 13 months of product usage data in atomic fact tables and creating an aggregated data table at the month, metric, namespace grain that only provides data for the past 13 months, an historical archive table would be able to provide insights from 2 or 3 years in the past for the aggregated table while the live data model would only provide the last 13 months of data.

SECTION: enterprise-data/platform/feast/_index.md

Feast is an open source feature store aimed to improve the process of obtaining, documenting and deploying features for data science (DS) models. In the DS GitLab team we are currently focused on using the Offline Store (used for batch non-realtime predictions). The repository for Feast can be found [HERE](#)

Note: Since Feast feature transformations capabilities are not yet mature enough and could not match the performance capabilities we need, we have opted for creating a separate fork of Feast where we optimize feature transformations based on our particular use case. You can read more about the fork, [HERE](#).

The most important concepts to understand the Feast Offline Store are:

1. Data Sources: The building blocks that contain the features that will be served. These can be specified as either a physical data source (CSV, parquet file, ...) or a SQL query. Since

Feast is currently lacking a feature transformation engine, we define any feature transformations as SQL queries.

2. Feature Views: Built with data sources and metadata of fields such as descriptions, tags, field type. Feature views are used when creating a new model, can be fetched through Feast.

3. Feature Services: Combination of fields from feature views that are needed to make inferences with the trained Machine Learning models. Feature services are used at the end, when we know which features are needed in the final model.

mermaid

graph LR

A[Data Sources] --> B[Feature Views]

A ~~~|"Data from Snowflake transformed through SQL"| A

B ~~~|"Data ready to be fetched through Feast"| B

C ~~~|"Group of feature views that define a ML model (e.g. PtC, PtE, ...)"| C

B --> C[Feature Services]

Advantages of using a feature store

1. Reusable Feature Pipelines: Reduces time to build predictive models by enabling the creation of feature pipelines that can be shared and reused across different teams and projects.

2. Consistency in Feature Engineering: Ensures uniformity of features during both training and production stages, as features are engineered once and used in multiple contexts; reduces SQL redundancies

3. Point-in-Time Correctness: Automates the maintenance of historical data which helps with retrieving correct values at any specific point in the past, crucial for predictive accuracy.

4. User Interface (UI): Facilitate easier discovery and management of features, reducing time to model creation.

5. Model Feature Versioning: Implements version control for features, aiding in tracking changes and maintaining consistency across model iterations.

6. Online Feature Store: Supports real-time predictions with an online store, potentially valuable for future projects requiring immediate data processing.

Setup

Feast uses a registry that acts as a central catalog which stores the feature definitions and the necessary metadata. At GitLab we store the registry in Snowflake.

There are two ways of interacting with the Feature Store:

1. Access already created feature views and services

2. Creating new feature views / services locally and testing them in your own unique database/Feast registry before asking for it to be merged to the production registry

For both of these workflows you need a read access to the FEATURESTORE.FEAST schema in Snowflake, which can be granted through Access Request.

To connect to the feature store we use the following command in python / jupyter notebook:

python

```
from feast import FeatureStore
store = FeatureStore(fsyamlfile=[credentialyamlfile])
```

Here `credentialyamlfile` indicates the YAML file with the necessary credentials to connect to the feature store. There are currently 3 credential files that can be chosen (these can be found in the [Feast repository]):

1. `productionfeaturestore.yaml`: This one connects to the production registry stored in the Snowflake database `schema FEATURESTORE.FEAST`. This YAML is used by the models ran in production and/or trained by CI.
2. `productionlocalfeaturestore.yaml`: Similar to above, instead it uses your Snowflake credentials instead of production ones.
3. `staginglocalfeaturestore.yaml`: This one connects to your personal Snowflake database and uses the registry stored there. For example, `{YOURUSER}PROD.FEAST`. This yaml file is only needed if you intend to follow the second workflow.

Instruction

If you only want to interact with the Feature Store as described in the first option (fetching already made feature views / services), then follow only the first 3 steps. If you also want to create new and/or modify feature views and services, complete all of them:

1. Set up the environment variables in your `~/.zshrc` file. These are the variables that are referenced in the `[credentialyamlfiles]`:

```
console
Feast Development variables
export FEASTDEVSNOWFLAKEACCOUNT="GITLAB"
export FEASTDEVSNOWFLAKEUSER="{USER}@gitlab.com"
export FEASTDEVSNOWFLAKEAUTHENTICATOR="externalbrowser"
export FEASTDEVSNOWFLAKEROLE="{USER}"
export FEASTDEVSNOWFLAKEWAREHOUSE="DEVXS"
export FEASTDEVSNOWFLAKESHEMA="FEAST"
export FEASTDEVSNOWFLAKEDATABASE="{USER}PROD"
export FEASTPRODSNOWFLAKEDATABASE="FEATURESTORE"
export FEASTPRODSNOWFLAKESHEMA="FEAST"
export FEASTUSAGE=False
```

Where user correspond to your snowflake user. For example, JPEGUERO. Make sure `{USER}` is in uppercase.

2. Run the command: `source ~/.zshrc`
3. In Snowflake, run the following command: `CREATE OR REPLACE SCHEMA {USER}PROD.FEAST;`. For example: `CREATE OR REPLACE SCHEMA JPEGUEROPROD.FEAST;`
4. To run the repository locally install `pipenv`.
5. Clone the repository into the `~/repos` directory and `cd` into it.
6. Finally, run the following commands:

```
console
mkdir .venv
pipenv install
make run-feast-local
```

These commands set up the python virtual environment. If you would also like to run the sample workbook make sure to also install the dev dependencies:

```
console
pipenv install --dev
```

make run-feast-local will run feast apply which creates the necessary infrastructure to deploy the offline store by the feature definitions in the featurerepo directory.

After this, the modelfeatureservicemanager is ran. This creates feature services out of YAML file definitions. These definitions needs to be stored in the featurerepo/models directory to be picked up.

Running the UI locally

Currently we have the UI running in a VM that can be accessed by requesting permission to the Data Engineering team. In case you would like to run the UI locally, you can use one of these two commands:

1. make ui-local: This runs the UI locally against the production database / production registry. After running the command you can access it through the following <https://0.0.0.0:8889>.
2. make ui-staging: This runs the UI against your personal development database instead of the production database (for example, when creating a MR to introduce new features and you want to test locally).

Project Structure

text

```
... featurerepo
.  ... product          <- Feature definitions for product data.
|  |  ... queries      <- Helper SQL queries to be used in the product
data sources definitions.
.  ... marketing        <- Intermediate data that has been transformed.
|  |  ... queries      <- Helper SQL queries to be used in the marketing
data sources definitions.
.  ... ...              <- Same as above.
|  |  ... queries      <- Same as above.
|  |
|  ... models           <- Contains YAML files with definitions for
creation of feature services.
|  |
.  ... entities.py      <- Entity definitions for the feature store.
```

```

| |
· ... helpers.py          <- Helper functions to be used when building the
feature definitions.
· |
| ... productionfeaturestore.yaml    <- Feast yaml file that contains credentials to sign
in to database. To be used when deploying changes to production and reading features for
production models.
| ... productionlocalfeaturestore.yaml <- Feast yaml file that contains credentials to sign
in to database. To be used when querying the feature store locally.
| ... staginglocalfeaturestore.yaml  <- Feast yaml file that contains credentials to sign
in to database. To be used when testing changes to the feature store locally.
.
... modelfeatureservicemanager      <- Module to generate feature services from a YAML
file. Used to create feature services for DS models (PtC, PtE, ...)
.
... createviews                    <- Module that generates views out of the features
defined in Feast. Helpful for taking advantage of MonteCarlo table lineage.

```

Currently (V1) feature repository is split in different functional areas such as product data, marketing, sales, ... Each one of those directories contains the necessary SQL queries that are used to create the data sources and the python files that create the feature views and annotate the fields with metadata.

actionableinsight is one of the metadata fields that describes the feature, which is used to populate insights about scores. To learn how actionable insight features are used with a feature service, review `featurerepo.helpers.getactionableinsightdescriptions`.

Workflow

Building ML models with Feast

Workflow summary:

1. Go to the Feast UI and find the features views that you want to use to train your model. Example: `productstagebyaccount5periodunit` and `productstagechangemetrics1periodunit`
1. Copy the `productionfeaturestore.yaml`, `productionlocalfeaturestore.yaml` and `staginglocalfeaturestore.yaml` to your ML project repository root.
1. Access feature views using the function `getfieldsfromfeatureviews`
1. Note that our ML model might only need a fraction of the fields in the feature views for inference, therefore you can create feature services. To do that, in the model directory create a YAML file that defines the name of the feature service and the name of the feature views used together with the required fields. An example can be seen here for the PtC V4 model.
 1. An utility function that is helpful to create such YAMLs is `createyamlformodelfeatureservice`.
1. Create a MR in this project and add the feature service yaml to the models directory. Test that the feature service contains all the features that you need.
 1. Feast can be build in a personal database by following the above Setup section. It allows to do local testing before pushing the changes to production. Make sure to use the `staginglocalfeaturestore.yaml` when fetching the feature services when working with the MR

workflow.

1. Push the changes to the remote branch in the repository. Wait for the CI jobs clone-image (clones the Docker image from production so Feast can run) and clone-feature-store-db (clones the FEATURESTORE.FEAST schema from production to a dedicated MR database) to pass.

1. Test the changes introduced to the feature store by running the CI job test-on-mr. This job runs the command `feast apply` on the MR feature store database testing that the feature store can run after the changes introduced.

1. After merging the MR and the feature service is now in production, connect to the feature store using the `productionlocalfeaturestore.yaml` credential file or `productionfeaturestore.yaml` when running on CI. Going back to our python code, this would look like:

```
python
from feast import FeatureStore
if islocaldevelopment:
    store = FeatureStore(fsyamlfile='path/to/productionlocalfeaturestore.yaml')
else:
    store = FeatureStore(fsyamlfile='path/to/productionfeaturestore.yaml')
```

Sample jupyter notebook can be found [HERE](#).

Also, make sure to check out the current implementation of PtC, specifically `createprodmodels.ipynb` and `scoringcode.ipynb`, as it is a model fully created and moved to production CI using Feast.

From Data Sources to serving features

Per diagram at the top of the page, there are three stages to serve features in Feast:

- Data Sources
- Feature Views
- Feature Services

Defining Data Sources and feature transformations

There are two ways of accomplishing this

Using scaffold queries (old method - less efficient for bigger tables)

To address lacking transformations in Feast, following types of SQL queries are used in the GitLab Feast repository:

1. Simple aggregations such as SUM, AVG (with optional grouping by account id, opportunity id, etc)
1. Rolling / window aggregations - summing / averaging a certain value over a rolling window of past months / weeks / days. Example can be found in product usage period usage query, to create this type of aggregation:
 - a. Create a "scaffold" CTE, this is, for every distinct account in the base table that the data source is based on we create an entry for the aggregation date we intend to use. In this case this means take all the entities together with all the snapshotmonths.

b. Join the scaffold back to the base CTE, we join on the main entityid AND on the desired interval of aggregation date. In this case, we wanted a rolling window of 5 months, and this was accomplished with: base.snapshotmonth BETWEEN ADDMONTHS(scaffold.snapshotmonth, -5) AND scaffold.snapshotmonth.

1. Period over period transformation. A good example is product usage change metrics query. The pattern here is to define a base CTE where we have the table at the wanted aggregation period, then we create a period2 CTE which will be the data in the past. In this example we wanted to shift the data by 1 month in the past and this is accomplished with DATEADD('month', 1, snapshotmonth) AS snapshotmonth2 in the period2 CTE.

The data source table in Feast should have following format:

Entityid (e.g. CRMACCOUNTID)	eventtimestamp	metric (e.g. productusageuniqueactiveusers)
xxx	2023-01-01	44
xxx	2023-02-01	49

The main entityid in the data source is unique when combined with the eventtimestamp. The metric field is the result of the SQL transformation. In case of product usage period usage query this would be a CRM Account per eventtimestamp (here: monthly aggregated data), and the metric would be something like the window average of uniqueactiveusers in the periodunit of aggregation. If periodunit = 5 we would have that the record of 2023-01-01 is averaging uniqueactiveusers from 2022-09-01 to 2023-01-01, the record of 2023-02-01 averages from 2022-10-01 to 2023-03-02 etc

Using jinja templating

As explained before, we are using a fork of Feast to add additional features to Feast. Namely, adding the use of Jinja templating to speed up feature transformation, specifically transformations involving rolling / window aggregations.

In the previous approach to creating feature transformations that need to consider these window aggregations we would have a scaffold query like this:

```
sql
WITH base AS (
    SELECT
    FROM prod.workspacedatascience.monthlystageusagebyaccount
), dimdate AS (
    SELECT firstdayofmonth AS snapshotmonth
    FROM prod.common.dimdate
    WHERE dateactual <= CURRENTDATE
    AND dateactual >= '2021-02-01'::DATE
), scaffold AS (
```

```

SELECT DISTINCT
    base.dimcrmaccountid,
    dimdate.snapshotmonth
FROM base
CROSS JOIN dimdate
)

SELECT
    a.dimcrmaccountid,
    a.snapshotmonth AS productusagedate,
    --number of all time features used
    SUM(b.stagecreatealltimefeatures) AS stagecreatealltimefeaturescnt,

FROM scaffold a
LEFT JOIN base b
    ON a.dimcrmaccountid = b.dimcrmaccountid
    AND b.snapshotmonth BETWEEN ADDMONTHS(a.snapshotmonth, -{periodunit}) AND a.snapshotmonth
GROUP BY 1, 2, 3

```

to:

```

sql
-- USETEMPLATEWORKFLOW
WITH base AS (
    SELECT
        FROM prod.workspacedatascience.monthlystageusagebyaccount
        {% if validation %}
        LIMIT 100
        {% endif %}
)
SELECT
    a.dimcrmaccountid,

    {% if gethistoricalfeatures %} b."entitytimestamp"::DATE {% endif %}
    {% if validation %} '2024-05-01'::DATE {% endif %} -- just for validation, hardcore a date
    AS snapshotmonth,

    --number of all time features used
    SUM(a.stagecreatealltimefeatures) AS stagecreatealltimefeaturescnt

FROM base a
{% if gethistoricalfeatures %}
INNER JOIN "entitydataframe" b
    ON a.dimcrmaccountid = b.dimcrmaccountid
    AND a.snapshotmonth BETWEEN DATETRUNC('month', ADDMONTHS(b."entitytimestamp"::DATE, -6))
    AND DATETRUNC('month', ADDMONTHS(b."entitytimestamp"::DATE, -1))
{% endif %}
GROUP BY 1, 2

```

Notice how in the first query we need to create a scaffold of all combinations of the entity key (dimcrmaccountid).

In the second query, we can access the "entitydataframe" (a list of accounts/namespaces/... that we want to retrieve the features for) using the jinja variable gethistoricalfeatures. This part of the query will only be executed during feature retrieval (when using the feature store to train or score a model). This way, we only calculate the transformation at feature retrieval time for the entities (for example, customerids) and timeframe we care about, speeding the query process.

We also introduce the validation variable. This is because when running feast apply, Feast needs to validate that the query can be built. With this we can add logic to make sure that the date key (snapshotmonth) is present at validation time (as there is no entitydataframe at validation time). Also, it lets us select only a limited amount of rows in the base CTE to make the validation query even faster.

To let feast know it needs to use this jinja workflow you need to add a comment at the top of your SQL query, -- USETEMPLATEWORKFLOW.

You can read more about this in the Feast fork.

Parameters

Feast allows to create reusable queries that can be modified with parameters. It is useful when one model may require rolling 5 months for the product usage data, while the other - 3 months of the same product usage metrics.

1. periodunit: Queries with periodunit result in columns that indicate window of time (usually in months). When creating the feature view, the periodunit is substituted with provided value. For example, for product usage period usage query a feature view named productusagemetrics5periodunit was created by setting the {periodunit} value to 5 months, as seen here

1. TODO - not implemented yet - entity: To address various levels of aggregation, variable entity can be used, which allows to extract specific granularity from the table. For example, when working with product-related table that is unique on both account and namespace level - entity will allow to refer to either of those granularities.

Connecting Feast to MonteCarlo (MC)

In the efforts the rest of the data teams more aware of how the DS team is using the source data to create machine learning (ML) models we started a process to make MC aware of the data that feast creates to feed into ML models. The benefits being:

1. More integration of the DS team with the whole set of tools used by the rest of the team
2. Easier for AEs and DEs to understand how changes in a data model might affect a downstream ML model that it is powered from that data
3. Help the DS team be aware whenever there is an upstream fail in one of the tables that are critical to the DS model

Once MC is aware of the feature definitions that Feast has created, we can leverage MC table lineage tracking and notification system to accomplish the above.

To make this connection possible we have created a script in the Feast repository, `createviews`. This works by taking all the feature definitions stored in Feast (both Feature views and services) and creating views out of them in the `PROD.LINEAGETRACKING` schema. Since all these feature definitions reference snowflake tables, MC can then use its lineage tracking to achieve the above.

This script is automated and runs in Gitlab CI every time a new branch is merged into the main branch.

An example of part of the PtC table lineage can be seen here:

Future improvements

1. Implement feast apply to run transformations for only those feature views or services that were updated (especially when testing locally)
1. Move helper functions to the `gitlabdata` python package.
1. Create authentication functions to replace the credential yaml files and avoid copy-pasting between repositories. This functionality should be added to `gitlabdata` python package.

SECTION: enterprise-data/platform/infrastructure/_index.md

Overview

Data Infrastructure pages are available in our Internal GitLab Handbook.

<i class="fas fa-map-marked-alt fa-fw -text-purple"></i> Quick Links

- Airflow
- Data Image Project
- Monte Carlo
- Python Guide
- Snowflake Guide

SECTION: enterprise-data/platform/jupyter-guide/index.md

See related repository

Features

- Virtual environment install of JupyterLab with the most useful extensions pre-installed

- Common python DS/ML libraries (pandas, scikit-learn, sci-py, etc.)
- Natively connected to Snowflake using your dbt credentials. No login required!
- Git functionality: push and pull to Git repos natively within JupyterLab (requires ssh credentials)
- Run any python file or notebook either from your local machine or in a GitLab repo
- Support for running Jupyter in VS Code
- Need a feature you use but don't see? Let us know on [bt-data-science](#)

Getting Started

JupyterLab is configured to run in a virtual environment on your local machine. If you prefer not to setup a virtual environment, you can instead use the data science docker image with CUDA support.

When setting up JupyterLab, the following will happen:

- uv will create and manage a Python virtual environment in `.venv/` containing all required packages. uv is extremely fast and a great replacement tool for many common python tools (pipenv, pip-compile, install, etc.)
- A virtual python environment will be created using uv and the python version and packages (and their dependencies) defined in `pyproject.toml`
- JupyterLab will be built within the venv

Installation Instructions

Note: This setup is geared towards MacOS. The code may also work on Linux or Windows with some modifications.

1. Prerequisites - before installing please make sure the following are installed on your local machine:

- Python3.11+
- Pip3 (usually aliased as pip).
- Git
- Certain versions of MacOS may require Xcode Command Line Tools to be installed. From the command line, `xcode-select --install`

1. Clone the data-science repo to your local machine `git clone git@gitlab.com:gitlab-data/data-science.git`

1. Navigate to the directory: `cd data-science`

1. Execute the following command: `make setup`. This will do several things:

- Check, and if necessary, install/update brew, node.js, uv, and certain python packages on your local machine.
- Setup a virtual environment in the data-science directory

1. Launch Jupyter

- To launch JupyterLab: `make jupyter`
- To launch VS Code with a Jupyter Server: `make jupyter-vscode` then follow the terminal instructions

Running from Docker

Although we recommend running JupyterLab from a virtual environment, sometimes that is not

always possible. In those instances, we have created a docker image that can be used.

1. Pull the image `registry.gitlab.com/gitlab-data/data-science/datascienceimage:latest` into your container manager (we prefer Rancher Desktop)

1. Use the `docker-compose.yml` to launch JupyterLab. In your terminal, navigate to the location of the data-science repository on your local machine and type `make jupyter-docker`

1. You will need to manually copy and paste the URL shown in the terminal into your web browser to load JupyterLab

Mounting a local directory

By default, the local install will use the data-science folder as the root directory for JupyterLab. This is not terribly useful when your code, data, and notebooks are in other locations on your computer. To change, this you will need to create and modify a Jupyter Notebook config file:

1. Open terminal and navigate to the data-science repo, e.g. `cd repos/data-science`.

1. Run `./venv/bin/jupyter lab --generate-config` which generates `/Users/{yourusername}/.jupyter/jupyterlabconfig.py`.

1. Browse to the file location and open it in a text editor

1. Search for the following line in the file: `c.ServerApp.rootdir = "` and replace with `c.ServerApp.rootdir = '/the/path/to/other/folder/'`. If unsure, set the value to your repo directory (i.e. `c.ServerApp.rootdir = '/Users/{yourusername}/repos'`). Make sure you remove the `"` at the beginning of the line.

1. Make sure you use forward slashes in your path. Backslashes could be used if placed in double quotes, even if folder name contains spaces as such as `"\\{yourusername}\\Any Folder\\More Folders"`

1. Rerun `make jupyter` from the data-science directory and your root directory should now be changed to what you specified above.

Enabling Jupyter Templates

The data science team has created modeling templates that allow you to easily start building predictive models without writing python code from scratch. To enable these templates:

- In your `jupyterlabconfig.py` that you created as part of the Mounting a local directory, add the following lines, replacing `/Users/{yourusername}/repos/` with the path to the data-science/templates repo on your local machine:

```
py
```

```
c.JupyterLabTemplates.templatedirs = ['/Users/{yourusername}/repos/data-science/templates']
```

```
c.JupyterLabTemplates.includedefault = False
```

- Launch JupyterLab and you should see a new Template icon. Click the icon and select which template you would like to use.

Setting Up Jupyter Extensions

- The data-science repo comes with many useful JupyterLab extensions pre-installed, including git and execute time, and system monitor.
- To get the most out of these (and to avoid having to configure them every time you run the container), create the following file: `/Users/{yourusername}/.jupyter/lab/user-settings/@jupyterlab/notebook-extension/tracker.jupyterlab-settings`
- Within that file, paste the following and save:

```
json
{
  "codeCellConfig": {
    "codeFolding": true,
    "lineNumbers": true,
  },

  "recordTiming": true,
}
```

Updating the Virtual Environment

1. From the data-science repo, pull the latest changes to your local machine `git pull`
1. Optional: You may update the dependencies using the `make recompile` command, or add/remove dependencies using `make add-packages` and `make remove-packages`
1. Re-run `make setup`
1. Launch JupyterLab via `make jupyter` or `make jupyter-vscode`

Connecting to Snowflake

1. Make sure on your local machine you have setup `/Users/{yourusername}/.dbt/profiles.yml` file which does not include your password. `profiles.yml` must be placed in this directory in your "home" directory otherwise you will not be able to connect to Snowflake from your local machine. You can use the sample profile as a reference
1. Run through the `authexample` notebook in the repo to confirm that you have configured everything successfully. You will get a browser redirect to authenticate your Snowflake credentials through Okta.
1. If you get an error then likely Snowflake is not properly configured on your machine. Please refer to the Snowflake and dbt sections of the Data Onboarding Issue. It is likely that your `.dbt/profiles.yml` is not setup correctly.

Connecting to GitLab Model Experiments (MLFlow Integration)

1. In your GitLab profile, create a personal access token access token with API permissions
1. Ensure that Model experiments is toggled on under Settings -> General -> Visibility, project features, permissions for your project in GitLab
1. Locate the project id for your project under Settings -> General
1. On your local machine, you will need to create two new environment variables `MLFLOWTRACKINGTOKEN` and `MLFLOWTRACKINGURI`
 1. Open up your shell resource file (`.zshrc`, for example) in your local machine home directory.

1. Add the following line `export MLFLOWTRACKINGTOKEN="your-access-token"`
1. Add the following line `export MLFLOWTRACKINGURI="https://gitlab.com/api/v4/projects/{your-project-id}/ml/mlflow"`, but with your project id. Alternatively, you can also place this directly in your notebook.
1. Save the file
1. Source the file (i.e. `source ./zshrc`) or exit terminal and restart
1. Launch JupyterLab. You should now be able to initialize the experiment tracker with the `mlflow.settrackinguri(os.getenv('MLFLOWTRACKINGURI'))` command in JupyterLab

Note: If looking to connect to the Model Experiments when using CI, refer to Model Training Step-by-Step Instructions

Some interesting libraries included

Data & Model Management

- GitLabDS: Tools to quickly perform common EDA tasks
- MLFlow: Experiment tracking and model registry
- Feast: Open-source feature store
- Papermill: Parameterizing, executing, and analyzing Jupyter Notebooks
- interpret: Interpretable model development

Visualisation tools

- Plotly
- Seaborn
- ydata-profiling: Data profiling

ML libraries

- Scikit-Learn: Suite of commonly used algorithms
- AutoTS: Automated time series forecasting
- XGBoost + Optuna: Powerful black-box method with automated hyperparameter optimization
- Tensorflow and Keras: Deep learning and neural networks
- Lifelines: Survival analysis tools

SECTION: enterprise-data/platform/monte-carlo/index.md

What and why

Monte Carlo (MC) is our Data Observability tool and helps us deliver better results more efficiently.

The Data Team default for observing the status of the data is using Monte Carlo. Creating any tests (called monitors in Monte Carlo) are done via the UI of Monte Carlo and reported according to the notification strategy. On another iteration in the near future we plan to implement Monitors as Code and these tests will also be version controlled. Currently dbt still used for existing tests, there is no roadmap in place to migrate these to Monte Carlo.

How We Operate Monte Carlo

We use the data-pipelines Slack channel for MC platform related alerts.

We are planning on using the data-analytics Slack channel in the near future for model related alerts, as soon as we have implemented the full notification strategy for Monte Carlo.

This work is planned under this epic for F23Q3: Onboard Analytics Engineers to the Monte Carlo Tool

Monte Carlo is an integral part of our Daily Data Triage and will replace the TD Trusted Data Dashboards.

mermaid
graph TD

mc(Monte Carlo)
sf(Snowflake Data Warehouse)
de(GitLab Team Member)

mc --> |alerts| de
de --> |improves| sf
sf --> |observes| mc

The whole body of work covering the Monte Carlo rollout at GitLab falls under epic Rollout Data Observability Tool with 100% coverage of Tier 1 Tables to improve Trusted Data, Data Quality, and Data Team member efficiency, where the work breakdown has been done and issues have been created to reflect the necessary steps until we are up and running with Monte Carlo on production.

Logging In

Login to Monte Carlo is done via Okta. Go to <https://getmontecarlo.com/signin>.

The following screen appears upon login and after providing your email and clicking "Sign in with SSO", you should be redirected to your Okta login.

Please note, you need to login via SSO and not via username/password.

A runbook of how everything is technically set up can be found in the Monte Carlo Runbook.

The gist of it is that there is an Okta Group called okta-montecarlo-users that is maintained by the Data team and has the Monte Carlo app assigned to it.

In order to be able to access Monte Carlo via Okta by default, your user should be part of the okta-montecarlo-users group.

For that you should submit an AR (similar ARs: Example AR 1, Example AR 2) and assign it to Rigerta Demiri (@rigerta) or ping the data channel linking the AR.

Navigating the UI

Once logged in, you should be able to see the Monte Carlo Monitors dashboard with details on

the objects being monitored and several custom monitors that have already been set up.

You can create a new monitor or view existing monitor details, such as definition and schedule and any anomalies related to it.

Alternatively, you can also list all the incidents by clicking on the Incidents menu item on the top menu bar, you can search for a specific model by querying the Catalog view or check Pipelines for a detailed lineage information on how the data flows from the source to the production model.

Depending on the role assigned to your user (by default every user logging in via SSO is assigned a Viewer role), you might be able to see Settings and check existing users and integrations (such as Slack integration, Snowflake integration, dbt integration etc.)

If you need your role to be updated, you can reach out to anyone on the data platform team and they will be able to modify your existing role.

More information on navigating the UI can be found in the official Monte Carlo documentation.

Adding a New Monitor

Monte Carlo will be running volume, freshness and schema change monitors by default on all the objects it has access to.

However, these checks are based on update patterns the tool learns from the data and if you need a specific custom check that runs on a certain schedule, you might want to add a custom monitor for that.

The official Monte Carlo documentation on monitors can be found in the Monitors Overview guide.

We have one Monte Carlo Snowflake Integration in place, which has two separate connections to Snowflake.

The first connection is called snowflake and it operates on DATAOBSPWH1, an XS Snowflake Warehouse.

The second connection called snowflake large and it operates on DATAOBSPWHL, a L Snowflake Warehouse.

Please make sure to mindfully choose the connection that makes most sense for your new custom monitor when adding a new one.

Only choose to run the monitor on the large warehouse if this is really necessary for your custom SQL query to run in a reasonable amount of time and to prevent it from timing out.

Fine-Tuning an Existing Monitor

If you want to modify an existing monitor, depending on the type of monitor, you can modify different parts of it such as the schedule, the timestamp field to be taken into account & the alert condition.

Responding To A Slack Alert

Currently, when we are getting notifications on different Slack channels, we can already triage the issue via Slack by assigning a status to it choosing from: Fixed, Expected, Investigating, No action needed and False positive (No status is a default status by MonteCarlo). Once we start investigating and we have a finding, if we write a comment on Slack i